

CSS para el diseño y la presentación web

El diseño no es lo que se ve, es lo que funciona. CSS es el puente entre la intención del diseñador y la experiencia del usuario.

— Adaptado de Steve Jobs

Resultado de aprendizaje

Al concluir la semana 3, el estudiante diseña hojas de estilo CSS3 modernas aplicando metodologías de organización (BEM, mobile-first), sistemas de *layout* (Flexbox, Grid), preprocesadores (Sass) y buenas prácticas profesionales [34, 21].

3.1 Historia y evolución de CSS

CSS pasó en 30 años de un lenguaje minimalista para colores y fuentes a un sistema robusto de diseño y *layout* con soporte nativo para grids, container queries y propiedades personalizadas (`-variables`). Conocer las etapas de su evolución revela por qué hoy podemos prescindir de frameworks pesados como Bootstrap en muchos proyectos. La caja siguiente narra esa historia.

De CSS1 al motor de los *design systems* modernos

En 1994, **Håkon Wium Lie**, trabajando junto a Tim Berners-Lee en el CERN, propuso un mecanismo para separar la presentación del contenido. La idea era radical en su simplicidad: que los autores y los usuarios pudieran proponer estilos en *cascada*, donde el navegador combinara las reglas según prioridades. La propuesta se convertiría en **CSS Level 1** (1996), el primer estándar W3C de hojas de estilo.

Durante quince años CSS evolucionó lentamente. CSS2 (1998) añadió posicionamiento absoluto y *media types*. La explosión llegó con **CSS3** (modular, desde 2011 en adelante), que introdujo gradientes, transiciones, animaciones, transformaciones, `border-radius`, `box-shadow` y dos sistemas revolucionarios: **Flexbox** (2012) y **Grid** (2017). Hoy CSS es el motor de los *design systems* modernos: Material Design (Google), Fluent (Microsoft), Tailwind CSS, Bootstrap.

3.1.1 Tabla evolutiva de CSS

La especificación de CSS ha avanzado en módulos independientes desde 2011, lo que significa que distintas partes del estándar maduran a velocidades distintas. La tabla siguiente resume los hitos más relevantes para el desarrollador web 2026.

Tabla 3.1: Hitos en la evolución de CSS.

Año	Versión	Aporte
1996	CSS 1	Selectores básicos, colores, fuentes, márgenes.
1998	CSS 2	Posicionamiento (absolute, relative), z-index.
2004	CSS 2.1	Revisión de CSS 2 que es la usada en navegadores antiguos.
2011	CSS 3 (modular)	Transiciones, transformaciones, animaciones, gradientes.
2012	Flexbox CR	Layout unidimensional con propiedades de eje.
2017	CSS Grid CR	Layout bidimensional con áreas nombradas.
2020	Custom Properties	Variables CSS nativas (<code>-color: #006633</code>).
2022	Container Queries	Estilos basados en el contenedor padre, no en el viewport.
2023	<code>:has()</code> selector	Selector ñpadre tras décadas de espera.
2024	Cascade Layers	Capas declarativas para resolver especificidad.
2025	CSS Nesting nativo	Anidamiento estilo Sass sin preprocesador.

3.2 Fundamentos de CSS

Antes de discutir layouts modernos, conviene revisar los tres pilares de CSS: cómo se aplica el estilo, qué selectores existen y cómo funciona la cascada y la especificidad. Estas bases son la diferencia entre escribir CSS que mantiene el código bajo control y escribir CSS que se vuelve ingobernable.

3.2.1 Las tres formas de aplicar estilos

Existen tres mecanismos para aplicar estilo a elementos HTML, en orden de preferencia profesional. La enumeración siguiente los expone con su justificación.

Ejemplo corto comentado (las tres formas) (Listado 3.1):

Listing 3.1: Ejemplo de HTML

```

<!-- 1. EXTERNO: archivo .css separado. Es la forma profesional -->
<!-- porque permite cache del navegador y separa contenido/forma. -->
<link rel="stylesheet" href="estilos.css">

<!-- 2. INTERNO: <style> dentro de <head>. Util para una sola pagina -->
<!-- o para CSS critico que debe cargar sin peticion adicional. -->
<style>
  p { color: navy; } /* p = selector de tipo (todos los <p>) */
</style>

<!-- 3. INLINE: atributo style dentro del tag. Evitar en produccion -->
<!-- porque pulveriza la separacion de responsabilidades. -->
<p style="color:red">Texto rojo (inline, anti-patron).</p>

```

1. **Inline** (`style="..."`): evitar en producción.
2. **Interno** (`<style>` en `<head>`): útil para páginas pequeñas o casos especiales.
3. **Externo** (`<link rel="stylesheet">`): el modelo profesional. Permite caché del navegador y separación de responsabilidades.

3.2.2 Selectores principales

Un *selector* CSS es la expresión que decide a qué elementos del DOM se les aplica un conjunto de propiedades. CSS ofrece más de una docena de tipos de selectores con distinta *especificidad*. La tabla siguiente recoge los más usados.

Ejemplo corto comentado (selectores principales) (Listado 3.2):

Listing 3.2: Ejemplo de HTML

```

/* Selector de TIPO: todos los parrafos */

```

```

p          { line-height: 1.6; }
/* Selector de CLASE: elementos con class="destacado" */
.destacado { background: yellow; }
/* Selector de ID: el unico elemento con id="logo" */
#logo      { width: 120px; }
/* Selector DESCENDIENTE: <a> dentro de <nav> */
nav a      { text-decoration: none; }
/* Selector de ATRIBUTO: inputs de tipo email */
input[type="email"] { border: 1px solid blue; }
/* PSEUDO-CLASE: estado del elemento */
a:hover    { color: red; } /* Cuando el cursor pasa encima */

```

Tabla 3.2: Selectores CSS más usados.

Selector	Aplica a	Especificidad
*	Todos los elementos	0
p	Por tag	1
.clase	Por clase	10
#id	Por ID	100
a:hover	Pseudo-clase	11
p::first-letter	Pseudo-elemento	2
ul > li	Hijo directo	2
[type="email"]	Por atributo	11
:has(img)	Padre con descendiente (2023)	varía

3.2.3 Pseudo-clases CSS para feedback de validación

Las pseudo-clases relacionadas con formularios permiten dar feedback visual al usuario sin JavaScript:

- **:valid** — el campo cumple la validación.
- **:invalid** — el campo no cumple.
- **:user-valid** — el usuario ya interactuó y es válido (no marca el campo vacío al cargar).
- **:user-invalid** — el usuario interactuó y es inválido.
- **:required** — el campo es obligatorio.
- **:placeholder-shown** — mientras el placeholder es visible.
- **:focus-visible** — foco visible (solo navegación con teclado).

Ejemplo aplicable a formularios HTML5 (los formularios fueron vistos en la semana 2) (Listado 3.3):

```

input:user-invalid { border-color: red; }
input:user-valid   { border-color: green; }
input:focus-visible { outline: 3px solid #0066CC; }

```

Listing 3.3: Ejemplo de CSS

3.3 Sistemas modernos de layout

Históricamente, maquetar páginas con CSS fue una pesadilla: floats, posicionamientos hack, márgenes negativos. Desde 2017, Flexbox y CSS Grid resuelven la mayoría de los layouts de forma limpia y declarativa. Las dos subsecciones siguientes los explican con ejemplos visuales.

3.3.1 Flexbox: layout unidimensional

Flexbox organiza elementos en una dimensión (fila o columna) y resuelve problemas históricos de CSS como centrado vertical, distribución de espacio y orden visual independiente del HTML [53].

Ejemplo 3.1 (RESUELTO) — Navbar responsivo con Flexbox

Objetivo: crear una barra de navegación que se adapte a pantalla grande (horizontal) y pequeña (vertical).

IDE: IntelliJ IDEA Ultimate.

Paso 1: Crear archivo `navbar.html`. Véase el Listado 3.4.

```

1 <!DOCTYPE html>
2 <html lang="es-EC">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width,
6     initial-scale=1.0">
7   <title>Navbar Flexbox</title>
8   <link rel="stylesheet" href="navbar.css">
9 </head>
10 <body>
11   <nav class="navbar" aria-label="Principal">
12     <div class="logo">UTEQ</div>
13     <ul class="menu">
14       <li><a href="#">Inicio</a></li>
15       <li><a href="#">Carreras</a></li>
16       <li><a href="#">Investigacion</a></li>
17       <li><a href="#">Contacto</a></li>
18     </ul>
19   </nav>
20   <main><p>Contenido de la pagina.</p></main>
21 </body>
22 </html>

```

Listing 3.4: navbar.html

Paso 2: Crear `navbar.css`. Véase el Listado 3.5.

```

1 * { margin: 0; padding: 0; box-sizing: border-box; }
2 body { font-family: system-ui, sans-serif; }
3
4 .navbar {
5   /* activa Flexbox (layout unidimensional) */
6   display: flex;                                     /* activa
7     Flexbox (layout unidimensional) */
8   /* alineacion principal en el eje horizontal */
9   justify-content: space-between;                    /*
10     alineacion principal en el eje horizontal */
11   /* alineacion en el eje secundario (vertical) */
12   align-items: center;                               /*
13     alineacion en el eje secundario (vertical) */
14   background: #006633;
15   padding: 1rem 2rem;
16   /* permite saltos de linea en flex */
17   flex-wrap: wrap;                                  /* permite
18     saltos de linea en flex */

```

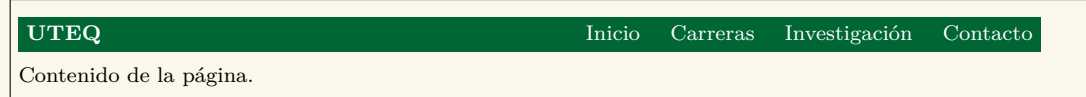
```

15 }
16
17 .logo {
18     color: white;
19     font-size: 1.5rem;
20     font-weight: bold;
21 }
22
23 .menu {
24     /* activa Flexbox (layout unidimensional) */
25     display: flex;                                /* activa
26     Flexbox (layout unidimensional) */
27     list-style: none;
28     /* espacio entre elementos hijos */
29     gap: 1.5rem;                                    /* espacio
30     entre elementos hijos */
31 }
32
33 .menu a {
34     color: white;
35     text-decoration: none;
36     padding: 0.5rem 0;
37 }
38
39 .menu a:hover { color: #CC9900; }
40
41 /* media query (responsive) */
42 @media (max-width: 600px) {                        /* media
43     query (responsive) */
44     /* direccion principal del flex container */
45     /* direccion principal del flex container */
46     .menu { flex-direction: column; width: 100%; gap: 0.5rem; }
47 }
48
49 main { padding: 2rem; }

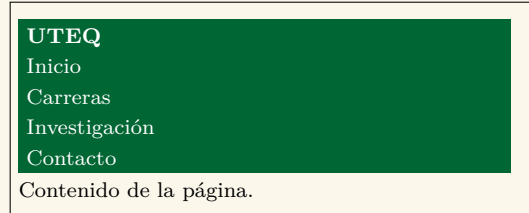
```

Listing 3.5: navbar.css

Resultado esperado (renderizado en pantalla ancha):



Resultado esperado (pantalla móvil < 600px):



3.3.2 CSS Grid: layout bidimensional

Grid permite layouts en dos dimensiones simultáneas. Es ideal para maquetar páginas completas, dashboards y galerías [34].

Ejemplo 3.2 (RESUELTO) — Galería de productos con Grid

Enunciado. Construir una galería de productos responsive con CSS Grid que se adapte automáticamente al ancho del viewport sin media queries explícitos.

```

1 .galeria {
2   /* activa CSS Grid (layout bidimensional) */
3   display: grid;                                /* activa
        CSS Grid (layout bidimensional) */
4   /* define columnas del grid */
5   /* define columnas del grid */
6   grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
7   /* espacio entre elementos hijos */
8   gap: 1rem;                                    /* espacio
        entre elementos hijos */
9 }
10 .producto {
11   border: 1px solid #ddd;
12   padding: 1rem;
13   border-radius: 8px;
14 }

```

Listing 3.6: galeria.css (fragmento)

Resultado esperado: una cuadrícula que ajusta automáticamente el número de columnas según el ancho del viewport (4 columnas en desktop, 2 en tablet, 1 en móvil).

Producto 1	Producto 2	Producto 3	Producto 4
\$2.50	\$3.20	\$4.50	\$1.80

3.4 Sass: el preprocesador profesional

Sass añade variables, mixins, anidamiento y módulos a CSS. Aunque CSS moderno ya tiene variables nativas, Sass sigue siendo el estándar en grandes proyectos por su capacidad de *computación*. Véase el Listado 3.7.

```

$verde: #006633;
$dorado: #CC9900;

@mixin boton($bg) {                                // definicion de
    mixin (codigo reutilizable)
    background: $bg;
    color: white;
    padding: 0.6rem 1.2rem;
    border: 0;
    border-radius: 4px;
    &:hover { filter: brightness(1.1); }           // referencia al
    selector padre
}

.btn-primario { @include boton($verde); }
.btn-acento   { @include boton($dorado); }

```

Listing 3.7: ejemplo.scss

3.5 Ejercicios

Los siguientes ejercicios consolidan Flexbox, Grid y Sass. El estudiante debe entregar el código fuente más una captura del resultado renderizado en el navegador.

Ejercicio 3.1 (PROPUESTO) — Card responsive con identidad UTEQ

Objetivo: construir un componente *card* reutilizable con HTML semántico + CSS Flexbox + Grid.

Requisitos:

- Tres cards alineadas con CSS Grid.
- Cada card tiene imagen (placeholder), título, descripción y botón.
- Colores institucionales (uteqverde #006633, uteqdorado #CC9900).
- Responsive: 3 columnas en desktop, 2 en tablet, 1 en móvil.
- Estado `:hover` con elevación (box-shadow).
- Score Lighthouse Accessibility ≥ 95 .

Entrega: archivos `cards.html` + `cards.css` con capturas en escritorio, tablet y móvil (3 imágenes).

Ejercicio 3.2 (PROPUESTO) — Dashboard básico con CSS Grid

Enunciado. Construir un dashboard básico con CSS Grid de 3 columnas que se reorganice a 1 columna en móviles. Crear un dashboard con cuatro tarjetas de estadísticas alineadas con Grid, una gráfica simulada con `<div>`+CSS y un menú lateral fijo. Aplicar mobile-first y validar accesibilidad.

3.6 Buenas prácticas CSS profesional

Conocer CSS no basta: la diferencia entre un proyecto mantenible y uno caótico está en aplicar disciplina al escribir estilos. La caja siguiente recopila doce reglas profesionales que el estudiante debe seguir desde el inicio.

Doce reglas CSS profesionales 2026

1. `box-sizing: border-box` global.
2. Mobile-first siempre.
3. Variables CSS para colores y tipografía (`-primario`).
4. BEM o similar para nombres de clase (`.card__titulo`).
5. Unidades relativas (`rem`, `em`, `%`) sobre `px`.
6. Sin `!important` excepto utilidades muy específicas.
7. No usar IDs como selectores (alta especificidad).
8. Flexbox para 1D, Grid para 2D.
9. Animaciones con `transform` y `opacity` (no afectan layout).
10. Respetar `prefers-reduced-motion` para usuarios sensibles a animaciones.
11. Lighthouse Performance + Accessibility ≥ 90 .
12. Minificación y autoprefixer en pipeline de build.